

Lab 5

This lab will cover content from the fifth week of the course, including:

- Selection
- Trees
- Feature Importance

Learning Objectives

- Interpret decision tree models to infer feature importances
- Compare feature importance information from regression models, PCA, and decision trees
- Visualize data in relevant spaces
- Select and justify use of supervised or unsupervised feature selection methods

Submission And Assignment Instructions

Submit your `Lab5_lastname_firstname.ipynb` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ❓ Questions you must answer or tasks are marked with a "❓"
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ❓ YOUR CODE HERE`
 - ⭐ Bonus questions are marked with a "⭐"
 - 🍓 Bonus answers are marked with a "🍓"

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Math - Node impurities

In the regression case, decision trees can minimize mean squared error between the predicted response value and the true response value. In the classification case, the "error" is slightly more complicated.

We're going to calculate some node impurities by hand! **Please show all your work for this section. Submit typeset or clear handwritten work.**

We're going to look at a toy dataset of patients classified as either sick or not sick, each of which has an associated weight and cough category (none, mild, or medium). In this scenario, we're trying to predict the patient's sickness based on their properties.

Weight (lbs)	Cough	Sick
155	Severe	Yes
200	None	Yes
132	Mild	No
92	None	Yes
186	None	No
160	Severe	No

? 1.1 Calculate the gini index (also called impurity) of the root node (all the samples).

✓ Answer:

```
In [2]: from IPython.display import Image, display
display(Image("1.1.png"))
```

$$1.1) \quad G_{root} = 1 - \sum p_i^2 = 1 - \left[\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 \right] = 0.5$$

? 1.2 Split by weight >150 lbs and calculate Gini index for the resulting.

✓ Answer:

```
In [3]: from IPython.display import Image, display
display(Image("1.2.png"))
```

1.2)

$$G_{\text{node A (weight > 150)}} = 1 - [(0.5)^2 + (0.5)^2] = 0.5$$

$$G_{\text{node B (weight ≤ 150)}} = 1 - [(0.5)^2 + (0.5)^2] = 0.5$$

$$G_{\text{split 1.2}} = \frac{4}{6}G_A + \frac{2}{6}G_B = \frac{4}{6}(0.5) + \frac{2}{6}(0.5) = 0.5$$

? 1.3 Split the patients by if cough exists or not and calculate Gini index for both resulting nodes.

✓ Answer:

In [4]: `from IPython.display import Image, display`
`display(Image("1.3.png"))`

1.3)

$$G_{\text{node A (cough exists)}} = 1 - \left[\left(\frac{1}{3} \right)^2 + \left(\frac{2}{3} \right)^2 \right] = 0.44$$

$$G_{\text{node B (cough does not exist)}} = 1 - \left[\left(\frac{2}{3} \right)^2 + \left(\frac{1}{3} \right)^2 \right] = 0.44$$

$$G_{\text{split 1.3}} = \frac{3}{6}(0.44) + \frac{3}{6}(0.44) = 0.44$$

? 1.4 Which split results in the greatest weighted average decrease in impurity?

✓ Answer: Given my calculation below, we can see that the when we split the patients by if cough exists or not, it results in the greatest decrease in impurity by 0.06. On the other hand, the first split by weight resulted in a 0 decrease.

In [25]: `from IPython.display import Image, display`
`display(Image("1,4.png"))`

$$1.4) \quad \Delta G_1 = G_{\text{root}} - G_{\text{split } 1.2} = 0.5 - 0.5 = 0$$

$$\Delta G_2 = G_{\text{root}} - G_{\text{split } 1.3} = 0.5 - 0.44 = 0.06$$

Section 2: Data Exploration

2.1: Load the Data

Complete the code blocks below to load the dataset and turn the chemical [SMILES strings](#) into interpretable molecular descriptors. This is an example of featurizing (digitalizing) molecules using [RDKit descriptors](#), which describe the given molecule using one of over 200 molecular properties.

The y-data for these molecules is the hemotoxicity of the molecule.

HemoTox dataset is from [here](#).

In [26]: `!pip install rdkit`

```
Requirement already satisfied: rdkit in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (2025.9.4)
Requirement already satisfied: numpy in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from rdkit) (1.26.4)
Requirement already satisfied: Pillow in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from rdkit) (9.4.0)
```

```
In [27]: import numpy as np
import pandas as pd
from rdkit import Chem
from rdkit.Chem import AllChem

#more datasets here: https://github.com/RekerLab/DeepDelta/tree/main/Datasets/
url = "https://raw.githubusercontent.com/RekerLab/DeepDelta/main/Datasets/BencI
c = pd.read_csv(url)

# look at the top of this dataset
#print(c.head())

# convert SMILES to molecules
mols = [Chem.MolFromSmiles(s) for s in c['SMILES']]

# create binary fingerprints for robust ML models
fps = np.array([np.array(AllChem.GetMorganFingerprintAsBitVect(m,2)) for m in mols])

# create descriptors for ML models with higher interpretability
from rdkit.Chem import Descriptors
descr = Descriptors._descList
calc = [x[1] for x in descr]
d_name = [x[0] for x in descr]
```

```
desc = np.array([np.array([f(m) for f in calc]) for m in mols])

# Convert the numpy array to a pandas DataFrame with descriptor names as columns
# extract y values from original dataframe into separate vector
# have a look at the top of this dataframe
a = np.isnan(fps).sum()
b = np.sum(np.var(fps, axis=0) == 0)
b1 = np.var(fps, axis=0)
print(c.shape)
print(fps.shape)
print("Missing values:", a)
print("Zero variance features:", b)
```

file:///Users/dianadayekh/Downloads/Lab5_Diana_Dayekh.html

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```
(828, 2)
(828, 2048)
Missing values: 0
Zero variance features: 427
```

It's important to consider how many observations (molecules) and features you have in any new dataset. With many features over a relatively small set of observations, overfitting is very easy.

? 2.2.1 How many observations and features are in this dataset? Are there missing (NA) observations? Are there any features with 0 variance (only 1 unique value)? Remove missing observations and 0-variance features from your dataset, reporting how many observations and features you removed. *Hint: You can use `VarianceThreshold` to remove features with 0 variance.*

```
In [29]: from sklearn.feature_selection import VarianceThreshold
## ? YOUR CODE HERE
selector = VarianceThreshold(threshold=0)
fps_clean = selector.fit_transform(fps)
print(fps_clean.shape)
a = np.isnan(fps).sum()
b = np.sum(np.var(fps, axis=0) == 0)
b1 = np.var(fps, axis=0)
print(c.shape)
print(fps.shape)
print("Missing values:", a)
print("Zero variance features:", b)
```

```
(828, 1621)
(828, 2)
(828, 2048)
Missing values: 0
Zero variance features: 427
```

✓ Answer: In the original dataset, there are 828 observations and 2048 features. There are 0 missing observations and 427 zero variance features.

Section 3: Regularized Multiple Linear Regression - the LASSO

Unlike Ridge regression or simple multiple linear regression, L1 regularization (the LASSO) can set coefficients of features to 0, effectively selecting a subset of features as "important" for the model.

? 3.1 Use the start of the code below to build a simple linear regression model to predict the output, using L1 regularization (LASSO). How many coefficients does this model set equal to 0? Print the top 10 coefficients (top by absolute value). A test-train split has been established for you. Initialize the LASSO model with `alpha=0.1, random_state=42`.

```
In [31]: from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.model_selection import train_test_split, cross_val_score

# Split the data into training and testing sets
X = fps_clean # features
y = c['Y'] # target
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42)
```

```

)

## ? YOUR CODE HERE

# Initialize the Lasso model
lasso = Lasso(alpha=0.1, random_state=42)

# fit the model
a = lasso.fit(X_train, y_train)

# Get the coefficients and their corresponding feature names
coef = a.coef_

# sort and display the top coefficients with their feature names
top_idx = np.argsort(np.abs(coef))[-10:]
# Print the top 10 features with the largest coefficients
for i in top_idx:
    print(f"Feature_{i}: {coef[i]}")

```

```

Feature_545: -0.0
Feature_692: -0.0016352197920019883
Feature_1099: -0.004931344224480191
Feature_741: -0.015363294990932516
Feature_1207: -0.016958758925654047
Feature_1045: -0.03270285437129713
Feature_810: -0.06572519343532705
Feature_1056: -0.08925426077337768
Feature_1524: -0.1516914288835764
Feature_812: -0.1994804499405401

```

How exciting! Our LASSO discarded a lot of features that it didn't find to be important. It could be that hemotoxicity is a function of just a few molecular features! Before we can conclude that, we should check to see if this model is actually a good fit for the data.

? 3.2 Does this model fit the data well? Make predictions on the test set and calculate the coefficient of determination (R^2) between the test predictions and test data. (1 sentence answer).

We'll revisit the LASSO to run cross-validation and compare to some decision trees later. This is just a first-pass check on one test data set.

```

In [32]: ## ? YOUR CODE HERE
# Predict on the test set using the fitted model
y_pred = a.predict(X_test)
# Calculate the R^2 value and print
# example of nice printing:)
r2 = r2_score(y_test, y_pred)
print("Coefficient of determination:", r2)

```

Coefficient of determination: 0.2570297013924068

✓ Answer: The model explains about 25% of the variance in the test data ($R^2 \approx 0.26$), indicating a relatively weak fit.

Section 4: Decision Trees

? 4.1 Write code below to build and evaluate a simple decision tree on all the data, using 5-fold cross-validation. Use `random_state = 12` to seed the Decision tree. Print the RMSE plus or minus its standard error (not standard deviation) from the cross-validation.

```
In [33]: from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_selection import VarianceThreshold
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

## ? YOUR CODE HERE
#NOTE FOR SELF: Each fold trains a separate tree / None of them are stored
# Initialize and train the decision tree regressor using 5-fold cross-validation

# X = fps_clean # features
# y = c["Y"] # target
# run cv on the full model
tree1 = DecisionTreeRegressor(random_state=12)
neg_mse_scores1 = cross_val_score(tree1, X, y, cv=5, scoring="neg_mean_squared_error")
# print RMSE (nice printing example)
mse_scores1 = -neg_mse_scores1
rmse_scores1 = np.sqrt(mse_scores1)
rmse_mean1 = rmse_scores1.mean()
rmse_se1 = rmse_scores1.std() / np.sqrt(len(rmse_scores1))

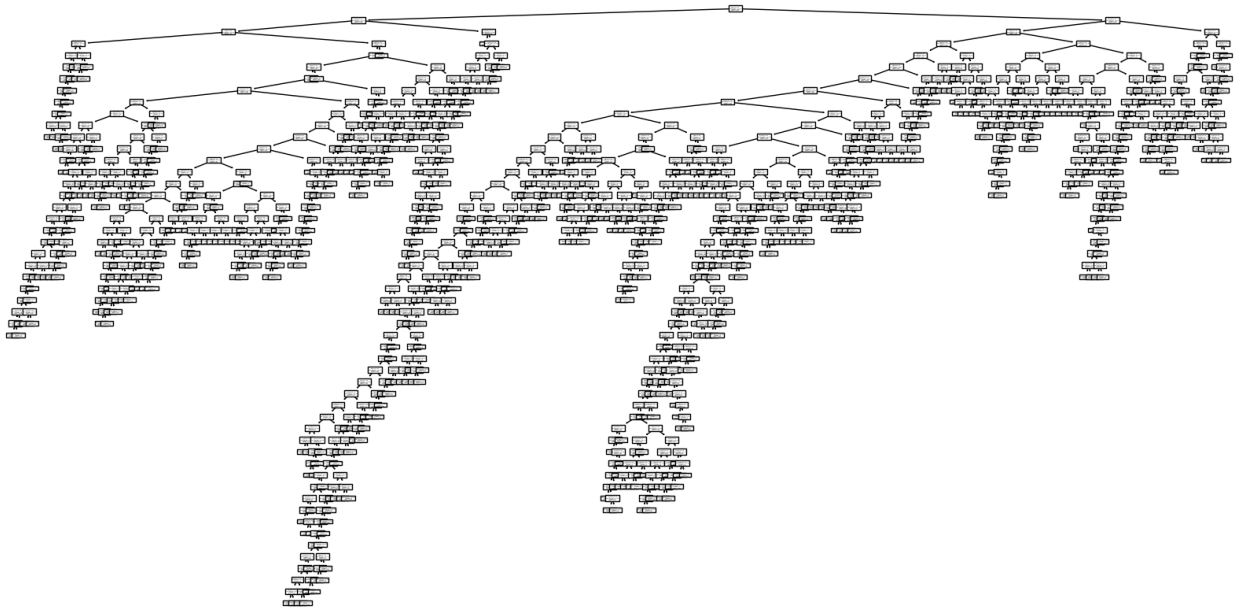
print(f"RMSE = {rmse_mean1:.3f} ± {rmse_se1:.3f}")

RMSE = 0.869 ± 0.056
```

If we try to visualize this model (using the code block below, which assumes your scaled dataframe is called `X_scaled`), you'll see that the very large number of features has made this model difficult to interpret, even if the tree organizes more important features to the top.

```
In [34]: # Train the model on the entire dataset
tree1.fit(X, y)

# Visualize the decision tree
plt.figure(figsize=(20,10)) #width #height
plot_tree(tree1)
plt.show()
```



? 4.2 Build a second model, this time with all the features but a reduced depth (use `random_state=12, max_depth = 4`). Report the 5-fold cross-validation RMSE, plus or minus its standard error (not standard deviation). *Start naming your models and RMSEs different things, you're going to have to compare them later.*

```
In [35]: ## ? YOUR CODE HERE
# Build a model with a limited depth
# cv on the shallow model, using all the data
tree2 = DecisionTreeRegressor(random_state=12, max_depth = 4)
neg_mse_scores2 = cross_val_score(tree2, X, y, cv=5, scoring="neg_mean_squared_
# print RMSE (nice printing example)
mse_scores2 = -neg_mse_scores2
rmse_scores2 = np.sqrt(mse_scores2)
rmse_mean2 = rmse_scores2.mean()
rmse_se2 = rmse_scores2.std() / np.sqrt(len(rmse_scores2))

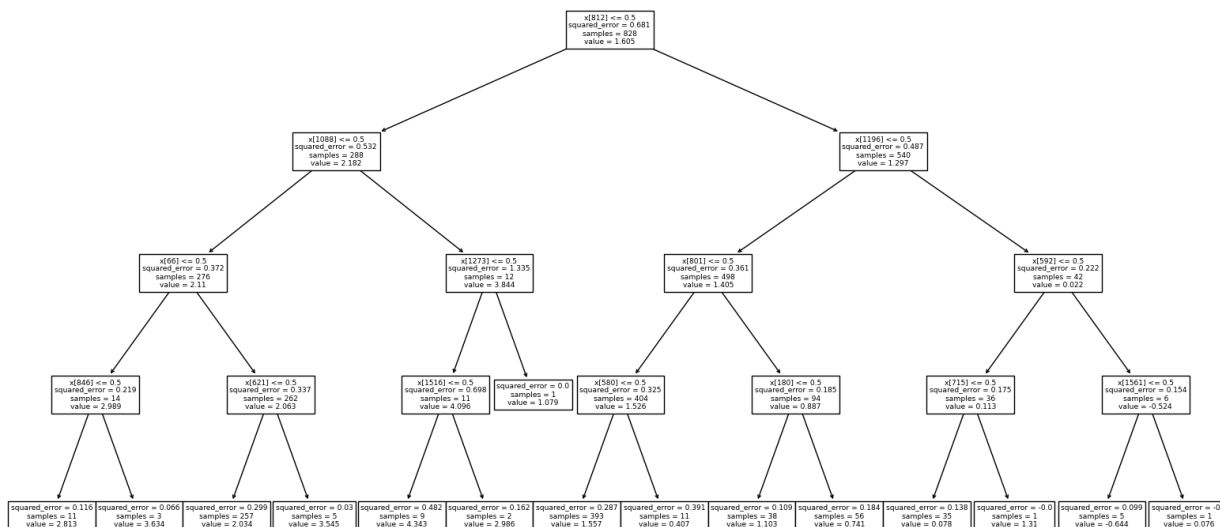
print(f"RMSE = {rmse_mean2:.3f} ± {rmse_se2:.3f}")
```

RMSE = 0.781 ± 0.102

Note that this new model only uses some of the features because it is depth-limited. This limitation also makes it much easier to visualize! The codeblock below will run and show your decision tree, provided you've called your new model `model_shallow`.

```
In [36]: # fit the model on the entire dataset (not cross-validation)
tree2.fit(X, y)

# Visualize the decision tree
plt.figure(figsize=(20,10)) #width #height
plot_tree(tree2)
plt.show()
```

? 4.3 Why does this second model have a lower cross-validation error? (1-sentence answer)

✓ Answer: When you reduce the depth of a tree, called tree pruning, the model tends to overfit less on the training data which is why we have a lower cross-validation error.

? 4.4 What are two advantages and two disadvantages to using decision trees?

✓ Answer:

Advantages:

- Easy to interpret
- Decision trees can reveal natural groupings in a dataset by splitting the data into distinct regions based on feature thresholds.

Disadvantages:

- Hard to use on complex data in higher dimensions / can be inaccurate on complex problems
- Difficult to optimize

? 4.5 Build a model with a reduced input feature set, using a Variance Threshold of 0.2 to remove low-variance features. Continue limiting the depth of the tree with `random_state=12, max_depth = 4`. Report the 5-fold cross-validation RMSE, plus or minus its standard error (not standard deviation).

you can use `selected_features = selector.get_support(indices=True)` to get the feature names of the of the data left after selection.

```
In [37]: ## ? YOUR CODE HERE
# Build a second model with low-variance features removed
# Initialize and train the reduced model using 5-fold cross-validation
# cv on the reduced model, using all the data
selector = VarianceThreshold(threshold=0.2)
X_reduced = selector.fit_transform(X)

tree3 = DecisionTreeRegressor(random_state=12, max_depth=4)

neg_mse3 = cross_val_score(tree3, X_reduced, y, cv=5, scoring="neg_mean_squared_error")
rmse3 = np.sqrt(-neg_mse3)

rmse_mean3 = rmse3.mean()
rmse_se3 = rmse3.std(ddof=1) / np.sqrt(len(rmse3))

print(f"RMSE = {rmse_mean3:.3f} ± {rmse_se3:.3f}")

RMSE = 0.783 ± 0.078
```

? 4.6 Use 5-fold cross-validation to compute RMSE and evaluate your LASSO model from earlier. Report the 5-fold cross-validation RMSE, plus or minus its standard error (not standard deviation).

```
In [38]: ## ? YOUR CODE HERE
# cross-validation, lasso is already defined above
# Visualize the decision tree
neg_mse_lasso = cross_val_score(lasso, X, y, cv=5,
                                scoring="neg_mean_squared_error")

rmse_lasso = np.sqrt(-neg_mse_lasso)

lasso_mean = rmse_lasso.mean()
lasso_se = rmse_lasso.std(ddof=1) / np.sqrt(len(rmse_lasso))

print(f"RMSE = {lasso_mean:.3f} ± {lasso_se:.3f}")

RMSE = 0.761 ± 0.161
```

? 4.7 Plot and compare all 4 models' RMSE with error bars (standard error).

Your models should be the "Full Decision Tree", "Shallow Decision Tree", "Filtered Shallow Decision Tree" and "LASSO"

```
In [39]: ## ? YOUR CODE HERE
# fitting the last tree 3
tree3.fit(X_reduced, y)
# Compare the models' RMSEs with error bars

models = [
    "Full Decision Tree",
    "Shallow Decision Tree",
    "Filtered Shallow Decision Tree",
    "LASSO"
]

rmse_means = [
    rmse_mean1,
```

```

rmse_mean2,
rmse_mean3,
lasso_mean
]

rmse_errors = [
    rmse_se1,
    rmse_se2,
    rmse_se3,
    lasso_se
]

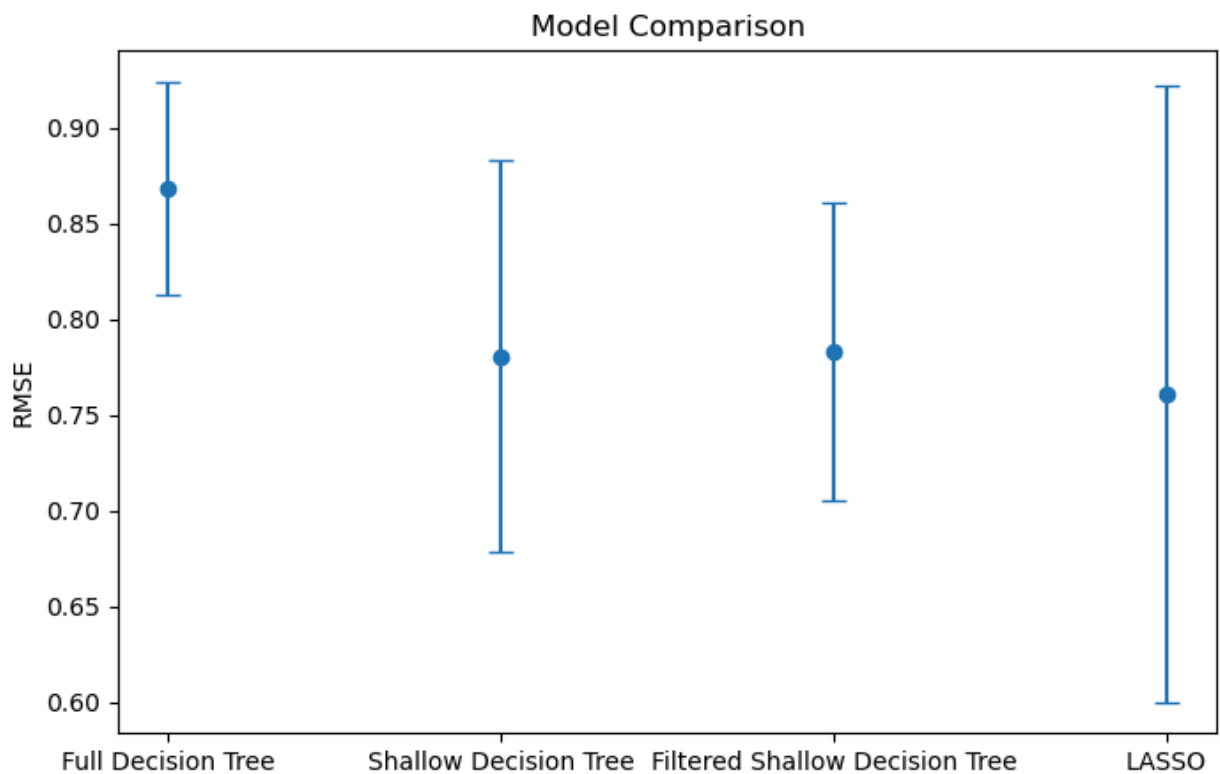
plt.figure(figsize=(8,5))

plt.errorbar(models, rmse_means, yerr=rmse_errors, fmt='o', capsize=5)

plt.ylabel("RMSE")
plt.title("Model Comparison")

plt.show()

```



? 4.8 Was there a significant change in model performance when you compare using all the data to using just a subset of the data (with the same `max_depth = 4` parameter)? Why or why not? *Hint: Both of these limited-depth trees have used "early stopping" to avoid overfitting. Generally, what variables are these trees using to make decisions?*

You should not have to do a statistical test to see whether or not there was a significant difference between these two conditions, it should be obvious from looking at your plot.

✓ Answer: The Full Decision Tree produced the highest RMSE, likely due to overfitting, which reduces its ability to generalize to new data.

Both the Shallow Decision Tree and the Filtered Shallow Decision Tree showed similar performance, as indicated by their overlapping error bars. This suggests that limiting the depth and removing less important variables did not meaningfully impact model accuracy, implying that many of the excluded features were not strongly predictive.

The LASSO model achieved a relatively low average RMSE, but its large error bars indicate higher variability across folds, suggesting less consistent performance compared to the shallow trees.

In []: